



Using the Legacy DTL Editor

Version 2026.1
2026-04-20

Using the Legacy DTL Editor

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

1 Introduction to the Legacy DTL Editor	1
1.1 A First Look at the Page	1
1.2 Introduction to the DTL Diagram	2
1.3 Controlling the Display	3
1.4 See Also	3
2 Creating Data Transformations (Legacy UI)	5
2.1 Building DTLs During a Migration	5
2.2 Creating a Transformation	5
2.3 Opening an Existing Transformation	6
2.4 Specifying Transformation Details	7
2.4.1 Using the Create existing Option	8
2.5 Undoing a Change	8
2.6 Saving a Transformation	8
2.7 Compiling a Transformation	8
2.8 Deleting a Transformation	9
2.9 See Also	9
3 Adding and Editing Actions (Legacy UI)	11
3.1 Adding an Action	11
3.2 Editing an Action	11
3.3 Rearranging Actions	12
3.4 Working with Groups of Actions	12
3.5 See Also	13
4 Details for Assign Actions (Legacy UI)	15
4.1 Introduction	15
4.1.1 Objects and Object References	15
4.2 Copying the Source Message	15
4.3 Copying a Value from a Source Property to a Target Property	16
4.4 Copying Values of All Sub-properties	17
4.5 Assigning a Literal Value to a Target Property	18
4.6 Using an Expression for the Value of a Target Property	18
4.6.1 Using the Data Transform Function Wizard	18
4.7 Assigning a Value to a Collection Item	19
4.8 Inserting a List Item	20
4.9 Appending a List Item	21
4.10 Removing a Property	22
4.11 Removing a Collection Item	22
4.12 Clearing a Collection Property	23
4.13 See Also	24
5 Details for Other Actions (Legacy UI)	25
5.1 Adding an If Action	25
5.2 Adding a For Each Action	26
5.2.1 Shortcuts for the For Each Action	27
5.2.2 Unloading Target Collections	28
5.2.3 Avoiding <STORE> Errors with Large Messages	28
5.3 Adding a Subtransform Action	29

5.4 Adding a Trace Action	30
5.5 Adding a Code Action	30
5.5.1 Guidelines for Using Custom Code in DTL	31
5.6 Adding an SQL Action	31
5.6.1 Guidelines for Using SQL in DTL	31
5.7 Adding a Switch Action	32
5.8 Adding a Case Action	32
5.9 Adding a Default Action	32
5.10 Adding a Break Action	32
5.11 Adding a Comment Action	33
5.12 See Also	33

1

Introduction to the Legacy DTL Editor

Note: Starting with 2025.1, the product includes a new DTL editor. To access this application, click **Try the new UI** on the existing DTL Editor. See [Introduction to the DTL Editor](#).

The legacy DTL Editor (the **Data Transformation Builder** page) enables you to create, edit, and compile DTL transformations. To access this page in the Management Portal, select **Interoperability > Build > Data Transformations**.

Important: After a period of inactivity, the InterSystems Management Portal may log you out and discard any unsaved changes. Inactivity is the time between calls to the InterSystems IRIS server. Not all actions constitute a call to the server. For example, clicking **Save** constitutes a call to the server, but typing in a text field does not. Consequently, if you are editing a data transformation, but have not clicked **Save** for longer than **Session Timeout** threshold, your session will expire and your unsaved changes will be discarded. After a logout, the login page appears or the current page is refreshed. For more information, see Automatic Logout Behavior in the Management Portal.

1.1 A First Look at the Page

When you display the DTL editor, it shows the last transformation you opened in this namespace, if any. This page has the following areas:

- The ribbon bar that the top displays options you can use to create and open DTL transformations, compile the currently displayed transformation, change the zoom display of the diagram, and so on.

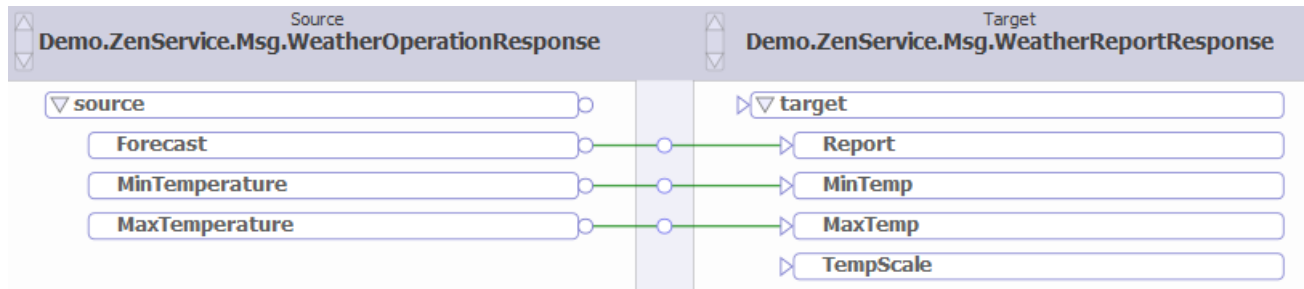
For information on these options, see [Creating Data Transformations \(Legacy UI\)](#).

- The upper part of the left area displays the [DTL diagram](#).
- The lower part of the left area displays a table that lists the actions defined in the DTL transformation. When InterSystems IRIS uses this transformation, it performs these actions in order as listed here.
- The right area displays three tabs:
 - **Transform**—Enables you to edit information about the transformation. For details, see [Specifying Transformation Details](#).
 - **Action**—Enables you to edit details of the selected action. Other topics describe the details for [assign actions](#) and for [other kinds of actions](#).
 - **Tools**—Enables you to launch a wizard to test the currently displayed transformation. For details, see [Testing Data Transformations](#).

- You can resize these three areas.

1.2 Introduction to the DTL Diagram

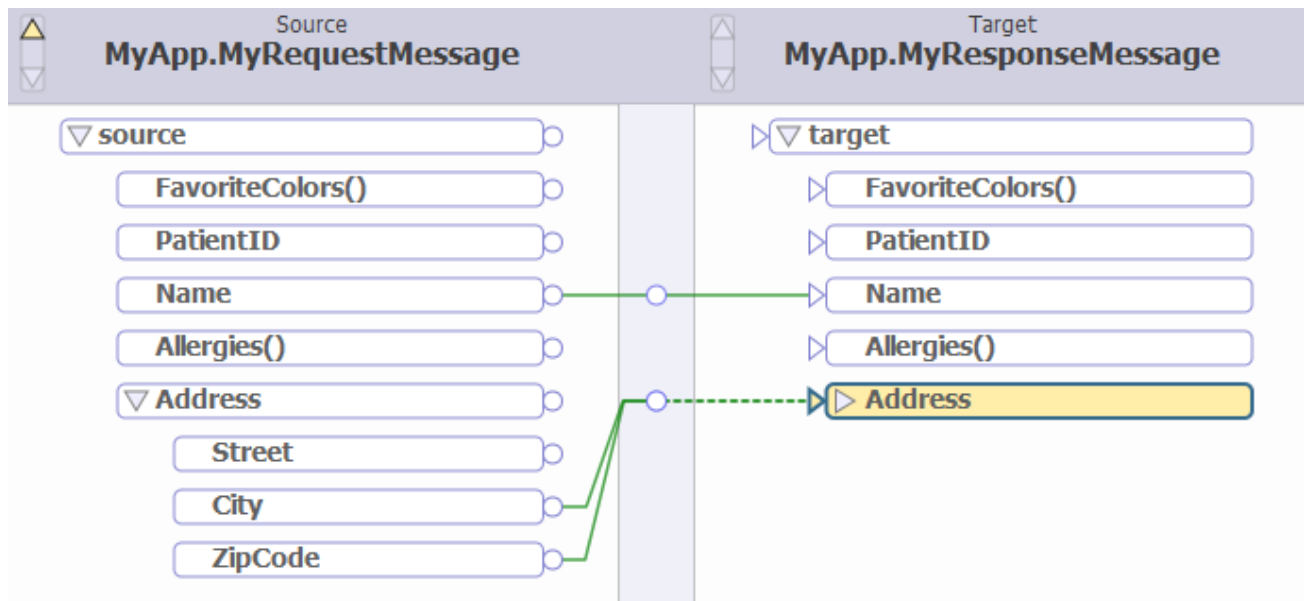
The following shows the DTL diagram for a DTL class:



Note the following points:

- The left area displays the source message. The header above the column displays the name of the source message class, and the boxes in the column display properties of the source message.
- The right area displays the target message in the same way.
- The top area includes a scroll button for each of these areas.
- The diagram shows connectors that represent actions within the transformations. The actions displayed here copy values from source properties to target properties.
- The center divider (the blue column) displays an icon on each connector line. The purpose of these icons is to enable you to select the connectors more easily. (You can select a connector line anywhere on its length, but it is easier to click the icons shown in this center divider.)

The following shows another example:



In this case, the source and target classes are more complex. Note the following additional points:

- The FavoriteColors property is defined as list of strings. This property is displayed here with parentheses () at the end of its name.

In this example, Allergies is another collection property.

- The Address property is defined as an object that has the Street, City, and ZipCode properties. Notice that the box for this property contains a triangle inside it.

In the left column, this property is displayed in expanded mode, so that you can see the properties. The triangle in the box is not solid and is pointing down.

The right column, this property is displayed in collapsed mode. The triangle in this box is solid and is pointing to the right.

- For the Address properties, the connector is shown with a dashed line on the side where the Address is collapsed. This indicates that there are hidden sub-properties on this side of the **assign** action.

1.3 Controlling the Display

You can control the display of the Data Transformation Builder page in multiple ways:

- You can click one of the **View** options in the ribbon bar:



Use the buttons to view both the transform diagram and the action list in the left pane of the page, or instead to collapse the section you do not want to see.

- You can select a zoom option from the drop-down list in the ribbon bar. By default, this list displays **100%**. Click a value in the list to shrink or enlarge the size of the DTL diagram.
- Use the scroll bars in the header area of the [DTL diagram](#).
- Collapse and expand the display of properties in the [DTL diagram](#).

1.4 See Also

- [Introduction to DTL Tools](#)
- [Creating Data Transformations \(Legacy UI\)](#)
- [Adding and Editing Actions \(Legacy UI\)](#)
- [DTL Syntax Rules](#)
- [Listing and Managing Data Transformations](#)
- [Testing Data Transformations](#)

2

Creating Data Transformations (Legacy UI)

This topic describes generally how to use the [legacy DTL editor](#) to create and edit [data transformations](#).

For information on performing these tasks with the new UI, see [Creating Data Transformations](#).

You can also visit Online Learning to try [Creating a Data Transformation](#).

2.1 Building DTLs During a Migration

If you intend to migrate to InterSystems IRIS for Health from other vendors, and you have a body of existing source and target messages, you can streamline the process of creating the necessary DTLs:

- You can automatically generate a starting DTL that performs the simpler transformations.
- You can compare the target messages output from the generated DTL to the original target messages, quickly identifying message segments that require attention.

To build a post-migration DTL using an existing body of source messages and target messages, follow the instructions at [DTL Generator](#).

2.2 Creating a Transformation

To create a transformation:

1. If you do not have the **Data Transformation Builder** page already open, then in the Management Portal, select **Interoperability > Build > Data Transformations**.
2. Click **New**.

If you are currently viewing a transformation and you have made changes but have not yet saved them, InterSystems IRIS® prompts you to confirm that you want to proceed (which will discard those changes).

InterSystems IRIS then displays a dialog box where you can specify the basic information for the transformation.

3. Specify some or all of the following information:

- **Package** (required)—Enter a package name or click the arrow to select a package in the current namespace. Do not use a reserved package name; see [Reserved Package Names](#).
- **Name** (required)—Enter a name for your data transformation class.
- **Description**—Enter an description for the data transformation; this becomes the class description.
- **Source Type** and **Source Class**—Specifies the type of messages that this transformation will receive as input.

Choose one of the following:

- **All Messages**—This transformation can be used with any input message type.
- **X12**—The input messages are instances of EnsLib.EDI.X12.Document.
- **EDIFACT**—The input messages are instances of EnsLib.EDI.EDIFACT.Document.
- **XML**—The input messages are instances of EnsLib.EDI.XML.Document.

Or click the search icon for **Source Class** and then select the class.

- **Source Document Type** (applicable only if the messages are virtual documents)—Enter or choose the document type of the source messages. You can choose any type defined in the applicable schemas loaded into this namespace.
- **Target Type** and **Target Class**—Specifies the type of messages that this transformation will generate as output. See the choices for **Source Type** and **Source Class**.
- **Target Document Type** (applicable only if the messages are virtual documents)—Enter or choose the document type of the target messages. You can choose any type defined in the applicable schemas loaded into this namespace.

Apart from **Package** and **Name**, you can edit all these details later.

4. Specify details on the **Transform** tab. See [Specifying Transformation Details](#).
5. Then [add actions](#) as needed.

2.3 Opening an Existing Transformation

To open a transformation:

1. Click **Open**.

If you are currently viewing a transformation and you have made changes but have not yet saved them, InterSystems IRIS prompts you to confirm that you want to proceed (which will discard those changes).

2. Click the package that contains the transformation.

Then click the subpackage as needed.

3. Click the transformation class.
4. Click **OK**.

2.4 Specifying Transformation Details

For a transformation, the Transform tab displays details that apply to the transformation as a whole. You may or may not have already specified some of these details. Other details can be edited only here. These details are as follows:

- **Name** (read-only)—Complete package and class name of the data transformation class.
- **Create**—Specifies how the transformation should create the target message. Choose one of the following:
 - **new**—Create a new object of the target class (and type, if applicable), before executing the elements within the data transformation. This is the default.
 - **copy**—Create a copy of the source object to use as the target object, before executing the elements within the transform.
 - **existing**—Use an existing object, provided by the caller of the data transformation, as the target object. See the [following subsection](#).
- **Source Class**—Specifies the type of messages that this transformation will receive as input. For details, see [Creating a Transformation](#).
- **Source Document Type** (applicable only if the messages are virtual documents)—Specifies the document type of the source messages.
- **Target Class**—Specifies the type of messages that this transformation will generate as output. For details, see [Creating a Transformation](#).
- **Target Document Type** (applicable only if the messages are virtual documents)—Specifies the document type of the target messages.
- **Language**—Specifies the language you will use in any expressions in this DTL. This can be **python** or **objectscript**.
- **Report Errors**—Specifies whether InterSystems IRIS should log any errors that it encounters when executing this transform. If you select this option, InterSystems IRIS logs the errors as *Warnings* in the Event Log. InterSystems IRIS also returns a composite status code containing all errors as its return value. This option is selected by default.
- **Ignore missing source segments and properties**—Specifies whether to ignore errors caused by attempts to get field values out of absent source segments of virtual documents or properties of objects. If you select this option, InterSystems IRIS suppresses these errors and does not call subtransforms where the named source is absent. This option is selected by default.

You can precisely control the behavior by including tests and conditional logic branches to confirm that any required elements are present.

- **Treat empty repeating fields as null**—Specifies whether InterSystems IRIS skips the following actions for repeating fields when the fields are empty:
 - **foreach** actions—If you select this option, InterSystems IRIS does not execute **foreach** actions on repeating fields that are empty.
 - **assign** actions—If you select this option, InterSystems IRIS does not execute **assign** actions on repeating fields if you use shortcut notation to indicate that both the **source** and **target** fields are repeating fields, and the **source** field is empty. For example, if the `source.{PV1:AdmittingDoctor() }` field is empty and you select this option, then InterSystems IRIS does not execute the following action:

```
<assign value='source.{PV1:AdmittingDoctor()}'
property='target.{PV1:AdmittingDoctor()}' action='set'.
```

However, InterSystems IRIS *does* execute the following similar action since the **target** field is not a repeating field:

```
<assign value='source.{PV1:AdmittingDoctor()}'  
property='target.{PV1:AdmittingDoctor(1)}' action='set' />
```

This option is cleared by default.

- **Description**—Description of the data transformation.

2.4.1 Using the Create existing Option

For **Create**, the **existing** option enables you to specify the target as an existing object, which results in a performance improvement. This option applies when you invoke a series of transformations programmatically (or perform other sequential processing). You would use this option in cases like the following scenario:

- You have three transformations that you want to perform in sequence:
 1. MyApp.ADTTransform—Uses the **new** option for **Create**.
 2. MyApp.MRNTransform—Uses the **existing** option for **Create**.
 3. MyApp.LabXTransform—Uses the **existing** option for **Create**.
- You invoke the transforms as follows:

```
do MyApp.ADTTransform.Transform(message, .target)  
do MyApp.MRNTransform(target, .newtarget)  
do MyApp.LabXTransform(newtarget, .outmessage)
```

2.5 Undoing a Change

To undo the previous change, click the Undo button .

2.6 Saving a Transformation

To save a transformation, do one of the following:

- Click **Save**.
- Click **Save As**. Then specify a new package, class name, and description and click **OK**.
- Click **Compile**. This option saves the transformation and then compiles it.

2.7 Compiling a Transformation

To compile a transformation, click **Compile**. This option saves the transformation and then compiles it.

2.8 Deleting a Transformation

To delete a transformation, you use a different page in the Management Portal:

1. In the Management Portal, click **Interoperability** > **List** > **Data Transformations**.
2. Click the row that displays its name.
3. Click the **Delete** button.
4. Click **OK** to confirm this action.

2.9 See Also

- [Introduction to the DTL Editor \(Legacy UI\)](#)
- [Adding and Editing Actions \(Legacy UI\)](#)
- [Listing and Managing Data Transformations](#)
- [Testing Data Transformations](#)

3

Adding and Editing Actions (Legacy UI)

A DTL transformation consists of a set of actions. The DTL editor provides options for adding, editing, and rearranging these, and this page describes generally how to use these options in the [legacy DTL editor](#).

For information on performing these tasks with the new UI, see [Adding and Editing Actions](#).

3.1 Adding an Action

To add an action, you can always do the following:

1. Optionally click a source or target property, depending on the kind of action you want to add.
2. Select an action from the **Add Action** drop-down list in the ribbon bar.
3. Edit the details for this action on the **Action** tab.

If applicable, the property that you selected is shown in the **Property** field, for use as a starting point. Optionally, you can disable the action with the **Disabled** check box. If you disable a **foreach** or **if** action, all actions within the block are also disabled.

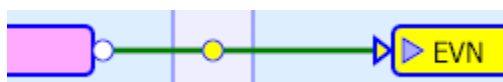
Other techniques are possible for **assign** actions, as discussed in [Details for Assign Actions \(Legacy UI\)](#).

3.2 Editing an Action

To edit an action, first select it. To do so:

- Click the corresponding row in the table below the diagram.
- If the diagram displays the action, click the icon on the corresponding connector line, in the center divider.

When you click on an item, it changes color, the connector line turns bold, and the source and target properties are highlighted in color. This means the connector is selected, as shown in the following figure.



Now edit the values on the **Action** tab. Optionally, you can disable the action with the **Disabled** check box. If you disable a **foreach** or **if** action, all actions within the block are also disabled.

Tip: If you double-click a property in the diagram, InterSystems IRIS updates the currently selected action, if applicable. If you double-click a field in the source, then the editor interprets it as your wanting to set the value for the selected action. Similarly, if you double-click a target field, the editor interprets it as your wanting to set the target for the selected action.

3.3 Rearranging Actions

InterSystems IRIS executes the actions in the order they are listed in the table below the diagram.

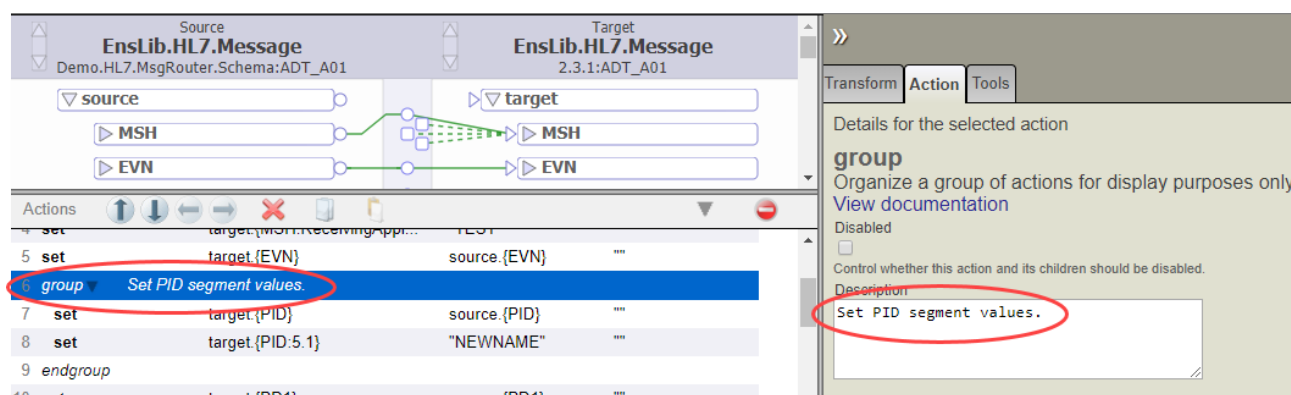
To rearrange actions, you must use the table below the DTL diagram, as follows:

1. Click the row corresponding to that action.
2. Click one of the following icons in that row, as needed:

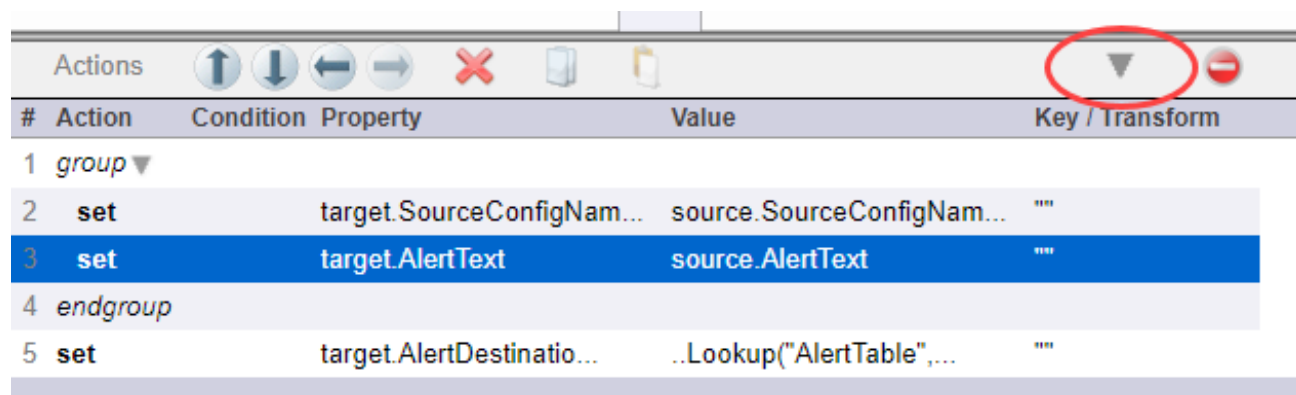
Tool	Description
	Move the selected action up one position. If the action is the first action in a group, for example a for each or if block, then this moves the action up and out of the group.
	Move the selected action down one position. If the action is the last action in a group, then this moves the action just after the group. For example, if the action is the last action in an if block, the action is moved right after the block. If the action is the last action in an if block just before the else , then this moves the action into the first position in the else block.
	Move the selected action out of the current group, for example a for each or if block. This moves the action out of the current group to the position immediately before the group.
	Move the selected action into the next group of actions, for example, a for each or if block.
	Remove all the actions of the data transformation.
	Remove the action in this row.

3.4 Working with Groups of Actions

You can gather actions into a display group by using the **group** action. Grouping actions helps organize them in the table below the diagram. The description that you define on the **Action** tab appears in the list to help you identify a group.



To move an action in or out of a group, select the action in the list and click  (move into group) or  (move out of group). To make the list more readable, you can collapse or expand groups as you review the list. To collapse a group and hide the actions it contains, click  next to the action name. To expand a group, click . You can also collapse and expand all groups at once using the  and  buttons in the table's **Actions** bar.



Note: You can also expand and collapse blocks of actions created by **if**, **for each**, **switch**, and **case** actions.

3.5 See Also

- [Introduction to the DTL Editor \(Legacy UI\)](#)
- [Creating Data Transformations \(Legacy UI\)](#)
- [Details for Assign Actions \(Legacy UI\)](#)
- [Details for Other Actions \(Legacy UI\)](#)

4

Details for Assign Actions (Legacy UI)

This topic provides details on adding different kinds of **assign** actions to DTL [data transformations](#), using the [legacy DTL editor](#).

For information on performing these tasks with the new UI, see [Actions That Set or Clear Values](#).

Important: For virtual documents, do not manually change escape sequences in the data transformation; InterSystems IRIS® handles these automatically.

4.1 Introduction

There are five kinds of **assign** actions: **set**, **clear**, **remove**, **append**, and **insert**. After you create any kind of **assign** action, you can change the type. To do so, select a different value in the **Action** drop-down in the **Action** tab.

InterSystems IRIS represents each **assign** action with a connector line in the [DTL diagram](#).

4.1.1 Objects and Object References

If you **assign** from the top-level source object or any object property of another object as your source, the target receives a cloned copy of the object rather than the object itself. This prevents inadvertent sharing of object references and saves the effort of generating cloned objects yourself. There is an exception: if the object has a property that is a list or array of objects, only the list reference is cloned, the actual objects within the list retain their original reference, thus still pointing to the source objects.

If you instead want to share object references between source and target, you must **assign** from the source to an intermediate temporary variable, and then **assign** from that variable to the target.

4.2 Copying the Source Message

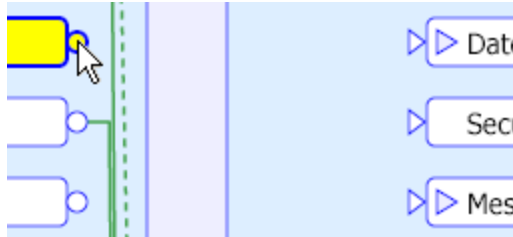
To create an **assign** action that copies the source message:

1. Drag the circle tab to the right of the source message. Hold down the mouse button.
2. Drag the cursor to the triangle tab to the left of the target message until its box changes color.
3. Release the left mouse button.

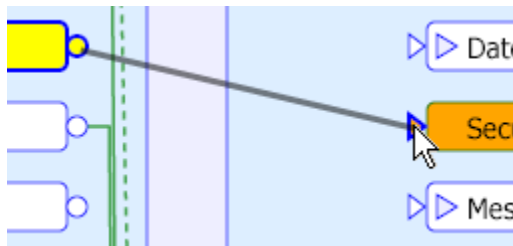
4.3 Copying a Value from a Source Property to a Target Property

To create an **assign** action that copies a value from a source property to a target property:

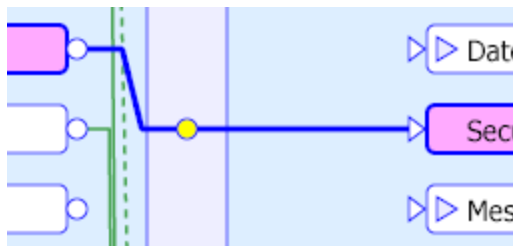
1. Drag a value from a source property to a target property. To do this:
 - a. Click the circle tab to the right of the source property. Hold down the mouse button. The display looks similar to this:



- b. Drag the cursor to the triangle tab to the left of the target property until its box changes color. The display looks similar to this:



- c. Release the left mouse button. The display looks similar to this:



The table below the diagram now shows the details of the assign action.

2. Optionally edit the details on the **Action** tab.

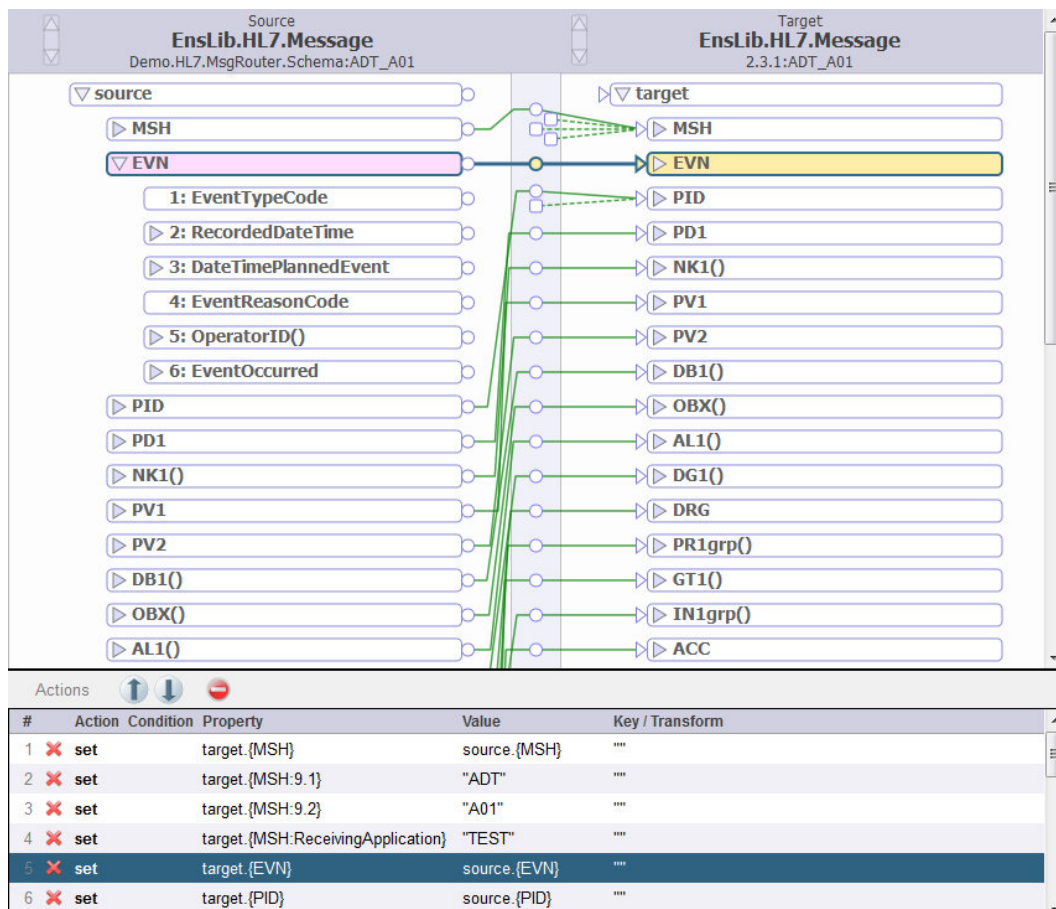
This **assign** action uses **set**.

4.4 Copying Values of All Sub-properties

If parent properties in the source and target are identical and the source and target have the same type, you can assign the values of all the sub-properties at once. In this case there is no need to expand the parent properties to reveal the sub-properties. Simply drag the cursor from the parent property on the source side to the parent property on the target side.

In the following DTL diagram, the single connector between the EVN property on the source side and the EVN property on the target side includes all of the following sub-properties of EVN:

- *EventTypeCode*
- *RecordedDateTime* and all of its sub-properties
- *DateTimePlannedEvent* and all of its sub-properties
- *EventReasonCode*
- *OperatorID*, all of its iterations, and all of its sub-properties
- *EventOccurred* and all of its sub-properties



This **assign** action uses **set**.

Note: If the source and target types are different, you cannot use this feature to assign subproperties, even if the structures appear to be parallel. For such a transformation, you must assign each leaf node of a structure independently and add a **for each** action to process iterations. See [Adding a For Each Action](#) for details on the **for each** action.

4.5 Assigning a Literal Value to a Target Property

To assign a literal value to a target property:

1. Select the target property.
2. Click **set** from the **Add Action** drop-down list. The **Action** tab for this operation displays.
3. Type a literal numeric or string value in the **Value** field:
 - A numeric literal is just a number. For example: 42.3
 - A string literal is a set of characters *enclosed by double quotes*. For example: "ABD"

Note: This string cannot include XML reserved characters. For virtual documents, this string cannot include separator characters used by that virtual document format. For details, see [DTL Syntax Rules](#).

4. Click **Save**.

4.6 Using an Expression for the Value of a Target Property

A literal value, as described in the [previous section](#), is a simple kind of expression. In some cases, you might want to use a more complex expression as the value of a target property. To do so, either:

- To create an expression that uses a function, click the search button  next to the **Value** field. This invokes the **Data Transform Function Wizard**, which is described in the following subsection.
 - To create a more complex expression, type the expression into the **Value** field.
- See [Valid Expressions](#). Make sure that the expression is valid in the scripting language you chose for the data transformation; see [Specifying Transformation Details](#).

4.6.1 Using the Data Transform Function Wizard

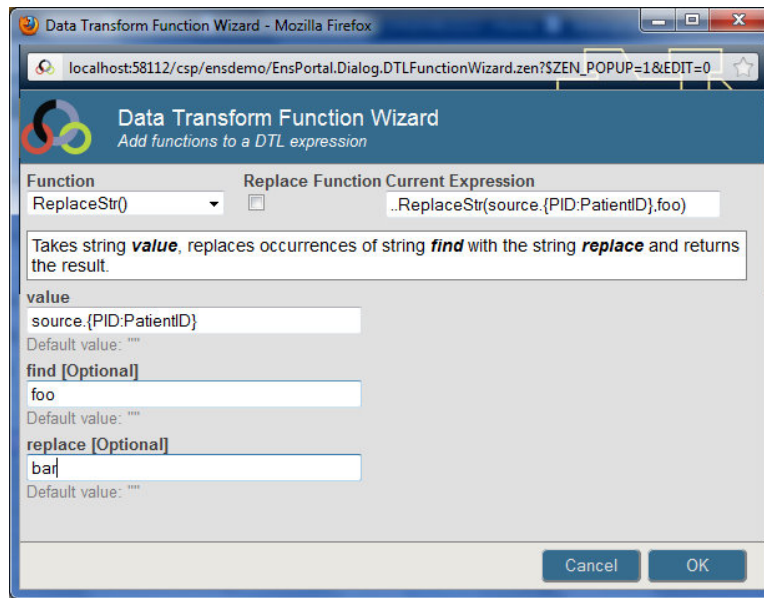
To use the **Data Transform Function Wizard**:

1. Select a **Function** from the drop-down list.

More fields display as needed to define the expression.

If you select **Repeat Current Function** from the drop-down list, a copy of the current function is inserted as a parameter of the itself, which creates a recursive call to the function.

2. Edit the fields as needed. For instructions, see the context-sensitive help in the dialog.
3. Click **OK** to save your changes and exit the wizard.



4.7 Assigning a Value to a Collection Item

This section applies to the following kinds of collections:

- Collection properties in standard production messages.
- Repeating fields in XML virtual documents.

To change the value of an item from a collection:

1. Select the target list property or array property.
2. Click **set** from the **Add Action** drop-down list. The **Action** tab for this operation displays.
3. In the **Property** field, edit the value in parentheses so that it identifies the item to change.

For array properties, use the key of the array item. For list properties, use the index of the list item. For repeating fields in virtual documents, use the index of the segment or field.

For example, suppose that you originally see this:

```
target.MyArrayProp.(1)
```

Edit the field to contain this instead:

```
target.MyArrayProp("key2")
```

4. Edit **Value** to contain a literal value or other valid expression.

See [Valid Expressions](#). Make sure that the expression is valid in the scripting language you chose for the data transformation; see [Specifying Transformation Details](#).

5. Click **Save**.

For example:

Action
set

Property
target.MyListProp.(3)
Property whose value will be set. Double-clicking a diagram will place that property in this field.

Value
"new value"
Value to assign to the property. Double-clicking a diagram will place that property in this field.

Or, edit the **Property** field to remove the trailing . (1) from the displayed value. Then use **key** to specify identify the item to change, as described in the [next section](#). For example:

Action
set

Property
target.MyListProp
Property whose value will be set. Double-clicking a diagram will place that property in this field.

Value
"new value"
Value to assign to the property. Double-clicking a diagram will place that property in this field.

Key
4
For collection properties, this string specifies the target of this assignment.

4.8 Inserting a List Item

This section applies to list properties (but not array properties) in standard production messages. You can also use this action with XML virtual documents; see [Routing XML Virtual Documents in Productions](#).

To insert an item into a list:

1. Select the target list property or array property.
2. Click **insert** from the **Add Action** drop-down list. The **Action** tab for this operation displays.
3. In the **Property** field, remove the trailing . (1) from the displayed value.

For example, suppose that you originally see this:

```
target.MyListProp.(1)
```

Edit the field to contain this instead:

```
target.MyListProp
```

4. Edit **Value** to contain a literal value or other valid expression.

See [Valid Expressions](#). Make sure that the expression is valid in the scripting language you chose for the data transformation; see [Specifying Transformation Details](#).

- For **key**, identify the index position for the new item.

For example:

5

- Click **Save**.

For example:

4.9 Appending a List Item

This section applies to list properties (but not array properties) in standard production messages. You can also use this action with XML virtual documents; see [Routing XML Virtual Documents in Productions](#).

To insert an item into a list:

- Select the target list property or array property.
- Click **append** from the **Add Action** drop-down list. The **Action** tab for this operation displays.
- In the **Property** field, remove the trailing **. (1)** from the displayed value.

For example, suppose that you originally see this:

```
target.MyListProp.(1)
```

Edit the field to contain this instead:

```
target.MyListProp
```

- Edit **Value** to contain a literal value or other valid expression.

See [Valid Expressions](#). Make sure that the expression is valid in the scripting language you chose for the data transformation; see [Specifying Transformation Details](#).

- Click **Save**.

For example:

Action
 append ▼

Property
 target.MyListProp
 Property whose value will be set. Double-clicking the diagram will place that property in this field.

Value
 "value to append"
 Value to assign to the property. Double-clicking the diagram will place that property in this field.

Key
 ""
 For collection properties, this string specifies the target of this assignment.

4.10 Removing a Property

This section applies to properties in virtual documents.

To remove a property:

1. Select the property.
2. Click **remove** from the **Add Action** drop-down list. The **Action** tab for this operation displays.
3. Click **Save**.

Important: When you remove properties from a virtual document, it is necessary to perform an additional step known as *building the map* for the message. There are two ways that you can do this:

- Before the steps that remove properties, set the `AutoBuildMap` property to build the map automatically when the properties are deleted. To do this, include a **set** action that sets `target.AutoBuildMap` equal to 1.
- After the steps that remove properties, call the **BuildMap()** method. To do this, include a **code** action that includes this line:

```
do target.BuildMap()
```

4.11 Removing a Collection Item

This section applies to collection properties (lists and arrays) in standard production messages. You can also use this action with XML virtual documents; see [Routing XML Virtual Documents in Productions](#).

To remove an item from a collection:

1. Select the target list property or array property.
2. Click **remove** from the **Add Action** drop-down list. The **Action** tab for this operation displays.
3. In the **Property** field, remove the trailing `. (1)` from the displayed value.

For example, suppose that you originally see this:

```
target.MyArrayProp.(1)
```

Edit the field to contain this instead:

```
target.MyArrayProp
```

4. For **key**, identify the item to remove.

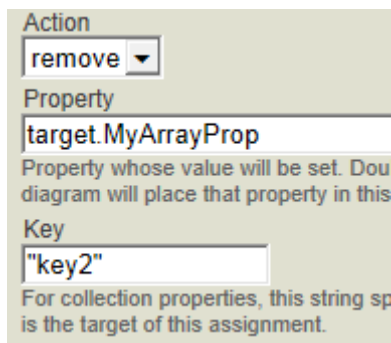
For array properties, use the key of the array item. For list properties, use the index of the list item. For repeating fields in virtual documents, use the index of the segment or field.

For example:

```
"key2"
```

5. Click **Save**.

For example:



Action

remove ▼

Property

target.MyArrayProp

Property whose value will be set. Diagram will place that property in this

Key

"key2"

For collection properties, this string specifies the target of this assignment.

4.12 Clearing a Collection Property

This section applies to collection properties (lists and arrays) in standard production messages. You can also use this action with XML virtual documents; see [Routing XML Virtual Documents in Productions](#).

To clear the contents of a collection:

1. Select the target list property or array property.
2. Click **clear** from the **Add Action** drop-down list. The **Action** tab for this operation displays.
3. In the **Property** field, remove the trailing . (1) from the displayed value.

For example, suppose that you originally see this:

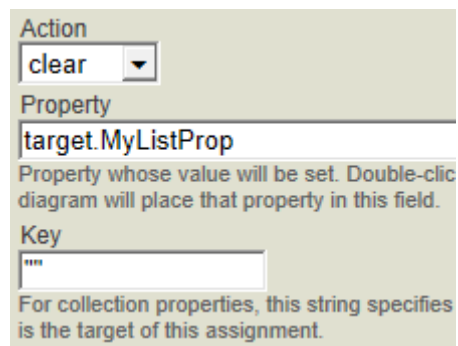
```
target.MyArrayProp.(1)
```

Edit the field to contain this instead:

```
target.MyArrayProp
```

4. Click **Save**.

For example:



Action
clear ▼

Property
target.MyListProp
Property whose value will be set. Double-click diagram will place that property in this field.

Key
"
For collection properties, this string specifies is the target of this assignment.

4.13 See Also

- [Adding and Editing Actions \(Legacy UI\)](#)
- [DTL Syntax Rules](#)
- [Details for Other Actions \(Legacy UI\)](#)
- [Testing Data Transformations](#)

5

Details for Other Actions (Legacy UI)

This topic provides details for DTL actions other than ([assign](#)), using the [legacy DTL editor](#).

For information on performing these tasks with the new UI, see [Other Actions](#).

5.1 Adding an If Action

An **if** action executes other actions conditionally, depending on the value of an expression that you provide. InterSystems IRIS[®] represents each **if** action as a connector line in the [DTL diagram](#).

To add an **if** action:

1. If the condition depends upon the value of a source property, click that property.
2. Select **if** from the **Add Action** drop-down list.

On the **Action** tab, the **Condition** field automatically contains the name of the source property that you had selected.

The area below the diagram contains three new rows. The Actions column displays the following labels for these rows:

- **if**—This row marks the beginning of actions to perform if the condition is true.
- **else**—This row marks the beginning of actions to perform if the condition is false.
- **endf**—This row marks the end of the **if** action.

3. Edit the **Condition** field so that it contains an expression that evaluates to either true or false.

For example:

```
source.ABC = "XYZ"
```

Notes:

- To create an expression that uses a function, click the search button  next to the **Value** field. This invokes the **Data Transform Function Wizard**, which is described [earlier](#).
- To create a more complex expression, type the expression into the **Value** field. See [Valid Expressions](#). Make sure that the expression is valid in the scripting language you chose for the data transformation; see [Specifying Transformation Details](#).

4. To add actions to perform when the condition is true:

- a. Click the **if** row.
- b. Select an item from the **Add Action** drop-down list.
- c. Edit the values in the **Action** tab as needed.
- d. Repeat as necessary.

You can include **assign** actions, **if** actions, and **for each** actions.

5. To add actions to perform when the condition is false:
 - a. Click the **else** row.
 - b. Continue as described in the preceding item.

The details are then shown in the block below the DTL diagram. For example:

6	✖	if	source.{PID:PatientIdentifierList(k1).id...	
7	✖	set	target.{PID:PatientIdentifierList(k1).as...	"AUSHIC" ""
8	✖	set	target.{PID:PatientIdentifierList(k1).id...	"MC" ""
9		else		
10		endif		

Note: It is not required to have any actions for the **if** branch or for the **else** branch. If there are no actions in either branch, the **if** action has no effect.

5.2 Adding a For Each Action

The **for each** action enables you to define a sequence actions that is executed iteratively, once for each member of one of the following:

- A collection property (for a standard message) .
- A repeating property (for a virtual document).
- A set of subdocuments in a document (for a virtual document).

InterSystems IRIS represents each **for each** action as a connector line in the [DTL diagram](#).

You can break out of a **for each** loop at any time by adding a **break** action within the loop.

To add a **for each** action:

1. Select a collection or repeating property in the source message.
2. Select **for each** from the **Add Action** drop-down list.

On the **Action** tab, the **Property** field automatically contains the name of the selected source property, and the **Key** field automatically contains k1, as illustrated below:

foreach
 Loop over the contents of a collection.
[View documentation](#)

Property
 source.{PID:PatientIdentifierList()}

Property to iterate over. Double-clicking on a property in this field.

Key
 k1

Name of the iterator variable used for this loop.

For the **for each** action, the **Key** field specifies the name of an iterator variable.

The **Property** field should not include the iterator key within the parentheses. For example, the following is correct:

```
source.{PID:PatientIdentifierList( )}
```

The **for each** iterates through the `PatientIdentifierList` repeating fields, starting with the first one (numbered 1) and ending with the last one.

- On the **Action** tab, the **Unload** check box controls whether to generate code to unload open objects or segments.

If the **unload** is checked for a **for each** action, then code is generated in the Transform method to try to unload/unswizzle open object(s) or segment(s) for the property collection at the end of each loop. Unsaved virtual document segments are saved and finalized. If the property is the source object, the source object is usually already saved.

You may still need to manually add actions to unload the target collection's objects or segments. For details on some strategies, see [Unloading Target Collections](#).

The unload of the **for each** property collection may be unnecessary – for example, for HL7, code generated using CopyValues does not instantiate the source segments.

- To add actions to the **for each** block, click the **for each** action and then add the appropriate actions.

The details are then shown in the block below the DTL diagram. For example (partial):

#	Action	Condition	Property	Value	Key / Transform
1	for each		source.{PID:PatientIdentifierList()}		k1
2	if	source.{PID:PatientIdentifierList(k1).id...			
3	set		target.{PID:PatientIdentifierList(k1).id...	"MR"	—
4	else				
5	endif				
6	if	source.{PID:PatientIdentifierList(k1).id...			

If the <foreach> applies to a collection property in a message, the sequence of activities is executed iteratively, once for every element that exists within the collection property. If the element is null, the sequence is not executed. The sequence is executed if the element has an empty value, that is, the separators are there but there is no value between them, but is not executed for a null value, that is, the message is terminated before the field is specified.

5.2.1 Shortcuts for the For Each Action

When you are working with virtual documents, InterSystems IRIS provides a shortcut notation that iterates through every instance of a repeating field within a document structure. This means you do not actually need to set up multiple nested **for each** loops to handle repeating fields; instead you create a single **assign** action using a virtual property path with empty parentheses within the curly bracket { } syntax. For information, see [Curly Bracket {} Syntax](#).

Note: If the source and target types are different, you cannot use this shortcut for the **for each** action. Use an explicit **for each** action in these cases.

5.2.2 Unloading Target Collections

While the **Unload** option automatically removes objects from a source collection, you need to add custom code at the end of a **for each** action to remove objects from a target collection. In a simple example in which the target is a complex record, you could use the following code to save the current target record and then unload it:

```
Do target.Record16.GetAt(k1).%Save(0)
Do target.Record16.%UnSwizzleAt(k1)
```

In other scenarios, it might be better to avoid loading the target altogether in order to avoid issues where the target is not unloaded. For example, suppose you have an object that has a parent/child property with many children. Within the **for each** action, you have a subtransform combined with *propSetObjectId(parentId)*, where *prop* is the name of the property.

In this example, the target is the batch object, the target class is `Demo.RecordMapBatch.Map.TrainDataOut.BatchOut` and the record class is `Demo.RecordMapBatch.Transform.Optimized.Record`

Before your **for each** loop, you need to create an empty target and assign its ID to a property `BatchOutID`:

```
<assign value='target.%Save()' property='tsc' action='set' />
<assign value='target.%Id()' property='BatchOutID' action='set' />
<assign value='target' property='' action='set' />
```

Then, in the **for each** loop, you can use code that directly impacts the target without having the target instantiated. For example:

```
<assign value=''' property='record' action='set' />
<subtransform class='Demo.RecordMapBatch.Transform.Optimized.Record' targetObj='record'
sourceObj='source.Records.(k1)' />

<comment>
<annotation>Assign record to target directly. </annotation>
</comment>
<assign value='record.%ParentBatchSetObjectId(BatchOutID)' property='tsc' action='set' />
<assign value='record.%Save()' property='tsc' action='set' />
```

Then, before the DTL ends, set the variable `target` back to the expected product of the DTL. For example:

```
<assign value='##class(Demo.RecordMapBatch.Map.TrainDataOut.BatchOut).%OpenId(BatchOutID)'
property='target' action='set' />
```

5.2.3 Avoiding <STORE> Errors with Large Messages

As you loop over segments in messages or object collections, they are brought into memory. If these objects consume all the memory assigned to the current process, you may get unexpected errors. You can avoid these errors in the source collection by using the **Unload** option in the Management Portal. For some strategies for removing objects in a target collection, see [Unloading Target Collections](#).

As another strategy, if you are processing many segments in a **for each** loop, you can call the **commitSegmentByPath()** method on both the source and target as the last step in the loop. Similarly, for object collections, use the **%UnSwizzleAt()** method.

The method **commitCollectionOpenSegments()** loops through the runtimePath looking for open segments within the specified collection path and calls **commitSegmentByPath()** for each open segment. This method is available from the classes `EnsLib.EDI.X12.Document`, `EnsLib.EDI.ASTM.Document`, `EnsLib.EDI.EDIFACT.Document`, and `EnsLib.HL7.Message`.

If you cannot make code changes, a temporary workaround is to increase the amount of memory allocated for each process. You can change this by setting the *bbsiz* parameter on the **Advanced Memory Settings** page in the Management Portal. Note that this action requires a system restart, and you should consult with your system administrator before performing it.

5.3 Adding a Subtransform Action

A **subtransform** invokes another transformation (an ordinary transformation), often within a **for each** loop. Subtransformations are particularly useful with virtual documents, because EDI formats are typically based on a set of segments that are used in many message types. The ability to reuse a transformation within another transformation means that you can create a reusable library of segment transformations that you can call as needed, without duplicating code transformation.

InterSystems IRIS does not represent a **subtransform** action in the [DTL diagram](#).

To add a **subtransform** action:

1. Select **subtransform** from the **Add Action** drop-down list.
2. On the **Action** tab, specify the following details:
 - **Transform Class**—Specifies the data transformation class to use. This can be either a DTL transformation or a custom transformation. For information on custom transformations, see [Defining Custom Transformations](#). You must enter the class.
 - **Source Property**—Identifies the property being transformed. This may be an object property or a virtual document property path. Generally it is a property of the source message used by the transformation. You must enter the source property.
 - **Target Property**—Identifies the property into which the transformed value will be written. This may be an object property or a virtual document property path. Generally it is a property of the target message used by the transformation. You must enter the target property.
 - **Auxiliary Property**—Optionally, specifies a value to be passed to the subtransform. The subtransform accesses the value as the *aux* variable. To pass multiple values:
 - a. Create an array variable with subscripts as in the following example:


```
set MyVar(1)="first value"
set MyVar(2)="second value"
```
 - b. Include a period immediately before the name of this variable, within the **Auxiliary Property** field. (The period indicates that this variable is passed by reference, which is the required way to pass a variable that has subscripts.)

Within the subtransform, you can access these values as `aux(1)` and `aux(2)`. That is, the *aux* variable has the same subscripts that you specified in the input array variable.
 - **Disabled**—Optionally, specifies that the subtransform is disabled.
 - **Description**—Optionally, specifies a text description of the subtransform.

Note: In the case of a **subtransform** with **Create** as **new** or **copy**, it is not necessary to have a pre-existing target object.

5.4 Adding a Trace Action

A **trace** action generates a trace message, which is helpful for diagnosis. If the [Log Trace Events](#) setting is enabled for the parent business host, this message is written to the Event Log. If the [Foreground](#) setting is enabled for the parent business host, the trace messages are *also* written to the Terminal window.

InterSystems IRIS does not represent a **trace** action in the [DTL diagram](#).

To add a **trace** action:

1. Select **trace** from the **Add Action** drop-down list.
2. On the **Action** tab, specify the following:
 - **Value**—Specify a literal value or other valid expression.
See [Valid Expressions](#). Make sure that the expression is valid in the scripting language you chose for the data transformation; see [Specifying Transformation Details](#).
 - **Description**—Specify an optional description.

The **trace** action generates trace message with User priority; the result is the same as using the \$\$\$TRACE macro in ObjectScript.

5.5 Adding a Code Action

A **code** action enables you to execute one or more lines of user-written code within a DTL data transformation. This option enables you to perform special tasks that are difficult to express using the DTL elements. InterSystems IRIS does not represent a **code** action in the [DTL diagram](#).

To add a **code** action:

1. Select **code** from the **Add Action** drop-down list.
2. On the **Action** tab, specify the following:
 - **Code**—Specify one or more lines of code in the scripting language specified for the transformation. For rules about expressions in this code, see [DTL Syntax Rules](#).

If you are using ObjectScript, make sure that each line starts with a space.

InterSystems IRIS automatically wraps your code within a CDATA block. This means that you do not have to escape special XML characters such as the apostrophe (') or the ampersand (&),

Also see the notes below.
 - **Description**—Specify an optional description.

Tip: To write custom code that you can debug easily, write the code within a class method or a routine so that it can be executed in the Terminal. Debug the code there. Then call the method or routine from within the code action of the DTL.

5.5.1 Guidelines for Using Custom Code in DTL

In order to ensure that execution of a data transformation can be suspended and restored, you should follow these guidelines when using a code action:

- The execution time should be short; custom code should not tie up the general execution of the data transformation.
- Do not allocate any system resources (such as taking out locks or opening devices) without releasing them within the same code action.
- If a code action starts a transaction, make sure that the same action ends the transactions in *all possible scenarios*; otherwise, the transaction can be left open indefinitely. This could prevent other processing or can cause significant downtime.

If you are using ObjectScript, make sure that each line starts with a space.

5.6 Adding an SQL Action

An **SQL** action enables you to execute an SQL **SELECT** statement from within the DTL transformation. InterSystems IRIS does not represent an **sql** action in the [DTL diagram](#).

To add an **sql** action:

1. Select **sql** from the **Add Action** drop-down list.
2. On the **Action** tab, specify the following:
 - **SQL**—Specify a valid SQL **SELECT** statement.
InterSystems IRIS automatically wraps your SQL within a **CDATA** block. This means that you do not have to escape special XML characters such as the apostrophe (') or the ampersand (&).
Also see the notes below.
 - **Description**—Specify an optional description.

5.6.1 Guidelines for Using SQL in DTL

Be sure to use the following guidelines:

- Always use the fully qualified name of the table, including both the SQL schema name and table name, as in:
`MyApp.PatientTable`
Where `MyApp` is the SQL schema name and `PatientTable` is the table name.
- Any tables listed in the **FROM** clause must either be stored within the local InterSystems IRIS database or linked to an external relational database using the SQL Gateway.
- Within the **INTO** and **WHERE** clauses of the SQL query, you can refer to a property of the source or target object. To do so, place a colon (:) in front of the property name. For example:

```
SELECT Name INTO :target.Name FROM MainFrame.EmployeeRecord WHERE SSN = :source.SSN AND City = :source.Home.City
```
- Only the first row returned by the query will be used. Make sure that the **WHERE** clause correctly specifies the desired row.

5.7 Adding a Switch Action

A **switch** action contains a sequence of one or more **case** actions and a **default** action. When a **switch** action is executed, it begins evaluating each **case** condition. When an expression evaluates to true, then the contents of the corresponding **case** block are executed; otherwise, the expression for the next **case** action is evaluated. As soon as one of the **case** actions is executed, the execution path of the transformation leaves the **switch** block without evaluating any other conditions. If no **case** condition is true, the contents of the **default** action are executed and then control leaves the **switch** block.

5.8 Adding a Case Action

Use the **case** action within a **switch** block to execute a block of actions when a condition is matched. When a **case** condition is met and the block of actions performed, the execution path of the transformation leaves the **switch** block without evaluating any other conditions.

To add a **case** action:

1. Select a **switch** action in the list below the diagram.
2. Select **case** from the **Add Action** drop-down list.
3. On the **Action** tab, add the condition. You can click the magnifying glass to add a function as part of the condition.
4. With the **case** action selected in the list below the diagram, use the **Add Action** drop-down to add the actions that will be executed if the condition evaluates to true.

5.9 Adding a Default Action

You cannot add a **default** block by using the **Add Action** drop-down list. Rather, the **default** action is automatically added to a **switch** block when you add the **switch** action. The actions contained in the **default** block are executed if none of the **case** conditions in the **switch** block are met. If you do not want anything to happen when none of the **case** conditions are met, simply leave the **default** block empty.

5.10 Adding a Break Action

Add a **break** action to a **for each** loop to leave the loop as soon as the **break** action is executed. After the **break** action is executed, the data transformation continues to process the action immediately following the **for each** loop.

If you add a **break** action outside of a **for each** loop, the data transformation terminates as soon as the **break** action is executed.

5.11 Adding a Comment Action

To help annotate the actions in a data transformation, you can add a comment that appears in the list of actions. After selecting **Add Action > Comment**, enter the comment in the **Description** text box on the **Action** tab.

5.12 See Also

- [Adding and Editing Actions \(Legacy UI\)](#)
- [DTL Syntax Rules](#)
- [Details for Assign Actions \(Legacy UI\)](#)
- [Testing Data Transformations](#)

